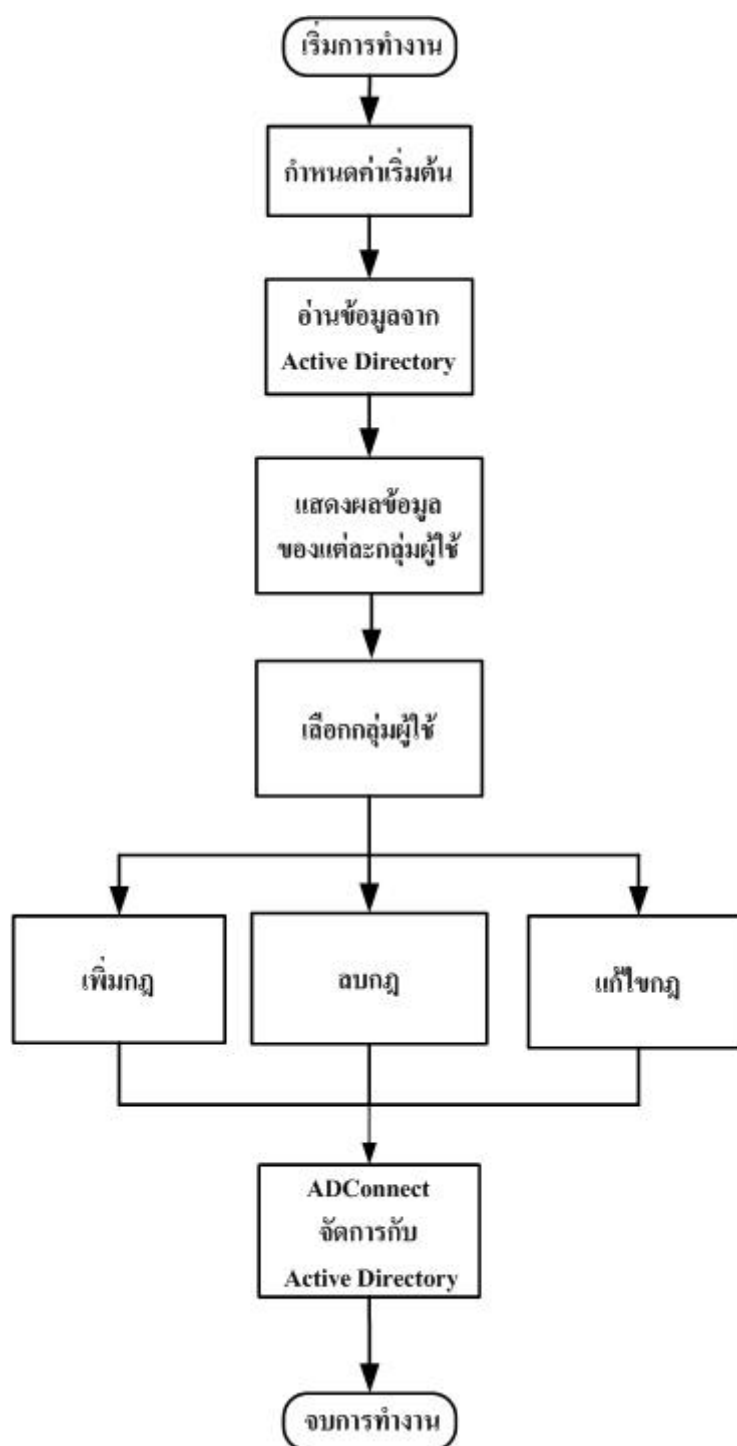


คู่มือโปรแกรมเมอร์ ชุดโปรแกรมเน็ตเวิร์กไฟร์วอลล์

นายจุมพล ทุมมาวัด
นายประภาส ผ่องสนาม
นายสุรศักดิ์ ทับเคลียว

ภาควิชาวิศวกรรมคอมพิวเตอร์
คณะวิศวกรรมศาสตร์
สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง
ปีการศึกษา 2547

คู่มือโปรแกรมเมอร์ โปรแกรม ISAG Firewall Administrator



รูปที่ 1 แสดงการทำงานของโปรแกรม Firewall Administrator

โปรแกรมนี้จะชื่อโปรเจ็คว่า FirewallManager มีคลาสทั้งหมด 5 คลาส คือ

1. ADConnect เป็นคลาสที่มีฟังก์ชันการติดต่อกับ Active Directory
2. CAboutDlg เป็นคลาสของ Dialog About ที่วิศวกร สร้างขึ้นมาอัตโนมัติ
3. CFirewallManagerApp เป็นคลาสที่จัดการเกี่ยวกับรูปแบบของโปรเจ็ค FirewallManager ที่วิศวกร (Wizard) สร้างขึ้นมาให้อัตโนมัติ
4. CFirewallManagerDlg เป็นคลาสที่รวมเอาฟังก์ชันการทำงานทั้งหมดของ โปรเจ็ค FirewallManager นี้ ที่เรากำหนดและสร้างขึ้นตามต้องการ
5. CRuleManage เป็นคลาสที่จัดการเกี่ยวกับกฎการฟิเตอร์ เช่น เพิ่ม ลบ และรีเซตกฎ เป็นต้น

และยังมีไฟล์ Header ที่สร้างเป็น Structure เพื่อเก็บพารามิเตอร์สำหรับกฎการฟิเตอร์ คือ rules.h จะประกอบด้วย

1. int action เซตค่าเป็น 0 เมื่อต้องการดรอ๊ป (Drop) แพ็กเกจ และเซตเป็น 1 เมื่อต้องการฟอร์เวิร์ด (Forward) แพ็กเกจ
2. unsigned long destinationIp เก็บค่าหมายเลขไอพีปลายทาง
3. unsigned long destinationMask เก็บค่าหมายเลขมาสก์ปลายทาง
4. unsigned short destinationPort เก็บค่าหมายเลขพอร์ตปลายทาง
5. unsigned int protocol เก็บค่าหมายเลขที่กำหนดเป็นโปรโตคอล
6. unsigned long sourceIp เก็บค่าหมายเลขไอพีต้นทาง
7. unsigned long sourceMask เก็บค่าหมายเลขมาสก์ต้นทาง
8. unsigned short sourcePort เก็บค่าหมายเลขพอร์ตต้นทาง

คลาส ADConnect

ประกอบด้วยฟังก์ชันสำคัญๆ ต่อไปนี้

1. CString GetDomain(void)
หน้าที่ : สำหรับเรียกเอาชื่อโดเมนของระบบ
พารามิเตอร์ : ไม่มี
ค่ารีเทิร์น : เป็นชื่อโดเมนแบบสตริง
2. bool GetList(CString View, CStringArray &Name, CStringArray &DistinguishedName)
หน้าที่ : สำหรับเรียกเอารายชื่อทั้งหมดที่ต้องการในกลุ่มที่ต้องการ

พารามิเตอร์ :

CString View เป็นชื่อกลุ่มที่ต้องการเรียกดูรายชื่อทั้งหมด เช่น group หรือ user เป็นต้น

CStringArray &Name เป็นรายการชื่อทั่วไปของรายการที่เรียกออกมาได้

CStringArray &DistinguishedName เป็นรายชื่อแบบระบบที่เรียกออกมาได้

ค่ารีเทิร์น : ค่าเป็น True หรือ False ว่าสามารถเรียกเอาค่าได้หรือไม่ได้

3. bool GetMember(CString GroupPath, CStringArray &Member)

หน้าที่ : เรียกเอารายชื่อของพาทกลุ่ม

พารามิเตอร์ :

CString GroupPath เป็นชื่อพาทกลุ่ม

CStringArray &Member รายชื่อกลุ่มที่เรียกออกมาได้

ค่ารีเทิร์น : ค่าเป็น True หรือ False ว่าสามารถเรียกเอาค่าได้หรือไม่ได้

4. bool GetMemberOf(CString UserPath, CStringArray &MemberOf)

หน้าที่ : เรียกเอารายชื่อของพาทยูซเซอร์

พารามิเตอร์ :

CString GroupPath เป็นชื่อพาทยูซเซอร์

CStringArray &Member รายชื่อยูซเซอร์ที่เรียกออกมาได้

ค่ารีเทิร์น : ค่าเป็น True หรือ False ว่าสามารถเรียกเอาค่าได้หรือไม่ได้

5. bool GetRule(CString Path, CStringArray &Rule)

หน้าที่ : เรียกเอารายการกฎออกมาจากกลุ่มที่ยูซเซอร์นั้นอยู่

พารามิเตอร์ :

CString Path เป็นพาทของกลุ่มที่ต้องการเรียกเอากฎการฟิเตอร์

CStringArray &Rule รายการกฎการฟิเตอร์ที่เรียกออกมาได้

ค่ารีเทิร์น : ค่าเป็น True หรือ False ว่าสามารถเรียกเอาค่าได้หรือไม่ได้

6. bool SetDeleteRule(CString Path, int Order)

หน้าที่ : ลบกฎการฟิเตอร์ออกจากกลุ่ม

พารามิเตอร์ :

CString Path เป็นพาทของกลุ่มที่ต้องการลบกฎการฟิเตอร์

int Order เป็นหมายเลขข้อของกฎที่ต้องการลบ

คำรีเทิร์น : ค่าเป็น True หรือ False ว่าสามารถเรียกเอาค่าได้หรือไม่ได้

7. bool SetAddRule(CString Path, CString NewRule)

หน้าที่ : เพิ่มกฎการฟิเตอร์ให้กลุ่ม

พารามิเตอร์ :

CString Path : เป็นพาทของกลุ่มที่ต้องการเพิ่มกฎการฟิเตอร์เข้าไป

CString Newrule : เป็นสตริงของกฎที่ต้องการเพิ่มเข้าไป

คำรีเทิร์น : ค่าเป็น True หรือ False ว่าสามารถเรียกเอาค่าได้หรือไม่ได้

คลาส CFirewallManagerDlg

มีฟังก์ชันสำคัญๆ ดังต่อไปนี้

1. void AddRuleToList(unsigned long srcIp, unsigned long srcMask, unsigned short srcPort,

unsigned long dstIp, unsigned long dstMask, unsigned short

dstPort,

unsigned int protocol, int action)

หน้าที่ : เพิ่มรายการกฎการฟิเตอร์เข้าไปแสดงผลที่ลิสรายการ

พารามิเตอร์ : พารามิเตอร์ต่างๆ จะมีความหมายเหมือนตัวแปรของไฟล์เฮดเดอร์ rules.h ที่กล่าวมาแล้วข้างต้น

คำรีเทิร์น : ไม่มี

2. void AddRule(void)

หน้าที่ : เพิ่มกฎการฟิเตอร์เข้าไปเก็บไว้ในระบบ

พารามิเตอร์ : ไม่มี (จะใช้ค่าต่างๆ จากตัวแปรของคลาส)

คำรีเทิร์น : ไม่มี

3. void BreakLongRule(CString sLongRule)

หน้าที่ : แยกกฎที่เป็นสตริงยาว ออกเป็นตัวแปรย่อยๆ

พารามิเตอร์ :

CString sLongRule เป็นค่าสตริงของกฎการฟิเตอร์แบบแถวเดียว
 คำรีเทิร์น : ไม่มี (โดยค่าที่แตกออกเป็นค่าตัวแปรย่อยๆ ของคลาส)

4. CString dwAddressToString(DWORD dwAddress)

หน้าที่ : เปลี่ยนค่าหมายเลขไอพีจากชนิด DWORD เป็นแบบ CString
 พารามิเตอร์ :

DWORD dwAddress เป็นหมายเลขไอพีชนิด DWORD

คำรีเทิร์น : หมายเลขไอพีชนิดสตริง

5. void GetDataFromAD(void)

หน้าที่ : เรียกเอาค่าต่างๆ ที่เริ่มต้นสำหรับการใช้งานจาก Active Directory

พารามิเตอร์ : ไม่มี

คำรีเทิร์น : ไม่มี (แต่จะได้ค่าหลายๆ ที่จำเป็นต้องใช้ Active Directory ที่ตัวแปรของ
 คลาส)

6. CString GetUserDN(void)

หน้าที่ : เรียกเอาชื่อของยูสเซอร์ที่ทำการล็อกอินอยู่ในขณะนั้น

พารามิเตอร์ : ไม่มี

คำรีเทิร์น : เป็นชื่อของยูสเซอร์ที่ล็อกอินอยู่ในขณะนั้นเป็นสตริง

7. void IPStringToIPBYTE(CString sIP)

หน้าที่ : แยกหมายเลขไอพีแบบออกเต็ตให้เป็นตัวแปรสี่ตัว

พารามิเตอร์ :

CString sIP เป็นหมายเลขไอพีแบบออกเต็ตแบบสตริงที่ต้องการแยก

คำรีเทิร์น : ไม่มี (จะถูกแยกไปที่ตัวแปรของคลาส)

คลาส CRuleManage

มีฟังก์ชันสำคัญๆ ดังต่อไปนี้

1. int AddRule(unsigned long srcIp, unsigned long srcMask, unsigned short srcPort,
 unsigned long dstIp, unsigned long dstMask, unsigned short dstPort,
 unsigned int protocol, int action)

หน้าที่ : เพิ่มกฎการฟิเตอร์เข้าไปยังตัวแปรที่เก็บกฎการฟิเตอร์

พารามิเตอร์ : พารามิเตอร์ต่างๆ จะมีความหมายเหมือนตัวแปรของไฟล์เฮดเดอร์ rules.h ที่กล่าวมาแล้วข้างต้น

ค่ารีเทิร์น : เป็นเลขจำนวนเต็ม 0 เมื่อถ้าเพิ่มกฎสมบูรณ์ และ จะเป็น -1 ถ้ามีข้อผิดพลาดในการเพิ่มกฎ

2. void DeleteRule(unsigned int position)

หน้าที่ : ลบกฎการฟิเตอร์ออกจากตัวแปรที่เก็บกฎการฟิเตอร์

พารามิเตอร์ :

unsigned int position เป็นเลขค่าของลำดับที่กฎที่ต้องการลบออกจากตัวแปรที่เก็บกฎการฟิเตอร์

ค่ารีเทิร์น : ไม่มี

3. static CString GetClientIP(void)

หน้าที่ : เรียกเอาหมายเลขไอพีของเครื่องนั้นๆ ขึ้นมา

พารามิเตอร์ : ไม่มี

ค่ารีเทิร์น : เป็นหมายเลขไอพีของเครื่องนั้นๆ แบบสตริง

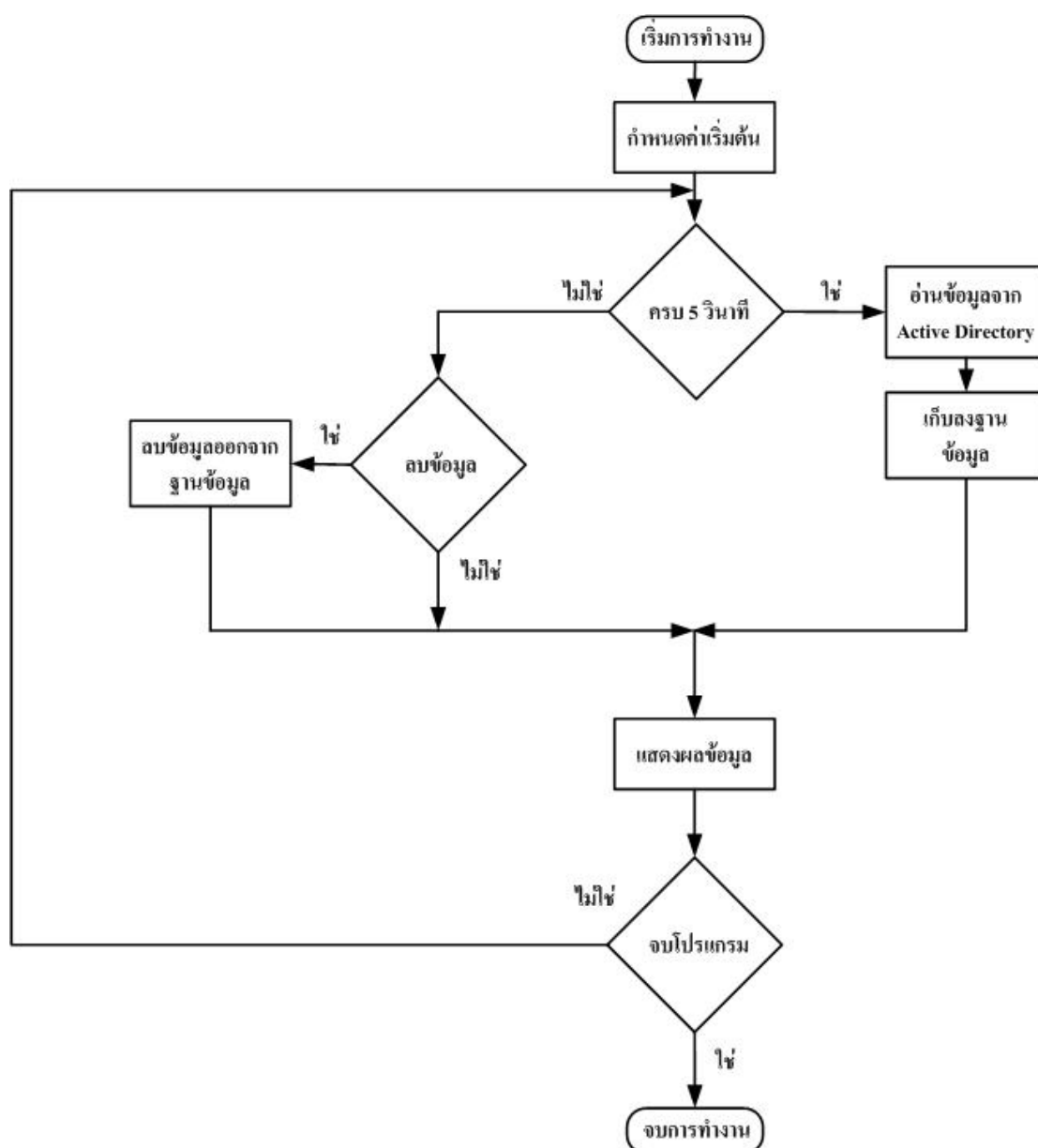
4. void ResetRules(void)

หน้าที่ : รีเซตจำนวนกฎการฟิเตอร์ที่เก็บอยู่ในตัวแปรที่เก็บกฎการฟิเตอร์

พารามิเตอร์ : ไม่มี

ค่ารีเทิร์น : ไม่มี

คู่มือโปรแกรมเมอร์ โปรแกรม ISAG Log Monitor



รูปที่ 2 แสดงการทำงานของโปรแกรม Log Monitor

โปรแกรมนี้จะชื่อโปรเจกต์ว่า LogMonitor มีคลาสทั้งหมด 5 คลาส คือ

1. ADConnect เป็นคลาสที่มีฟังก์ชันการติดต่อกับ Active Directory
2. CAboutDlg เป็นคลาสของ Dialog About ที่วิศวกร สร้างขึ้นมาอัตโนมัติ
3. CLogMonitorApp เป็นคลาสที่วิศวกร สร้างขึ้นมาอัตโนมัติ
4. CLogMonitorDoc เป็นคลาสที่วิศวกร สร้างขึ้นมาอัตโนมัติ

5. CLogMonitorSet เป็นคลาสที่วิซาร์ด สร้างขึ้นมาอัตโนมัติ
6. CLogMonitorView เป็นคลาสที่วิซาร์ด สร้างขึ้นมาอัตโนมัติ เราสามารถกำหนดสร้างฟังก์ชันใช้งานต่างๆ ที่คลาสนี้
7. CMainFrame เป็นคลาสที่วิซาร์ด สร้างขึ้นมาอัตโนมัติ

คลาส CLogMonitorView

มีฟังก์ชันที่สำคัญๆ ดังต่อไปนี้

1. void AddLogToDB(CString sLog)

หน้าที่ : จัดเก็บเอาล็อกไปเก็บไว้ที่ฐานข้อมูล

พารามิเตอร์ :

CString sLog เป็นข้อมูลการโจมตีที่อยู่ในรูปสตริงยาวๆ

ค่ารีเทิร์น : ไม่มี

2. void AddLogToList(int iAllRecord)

หน้าที่ : นำข้อมูลทั้งหมดที่อยู่ในฐานข้อมูลไปแสดงที่ลิสต์

พารามิเตอร์ :

int iAllRecord เป็นจำนวนของข้อมูลทั้งหมดที่เก็บอยู่ในฐานข้อมูล

ค่ารีเทิร์น : ไม่มี

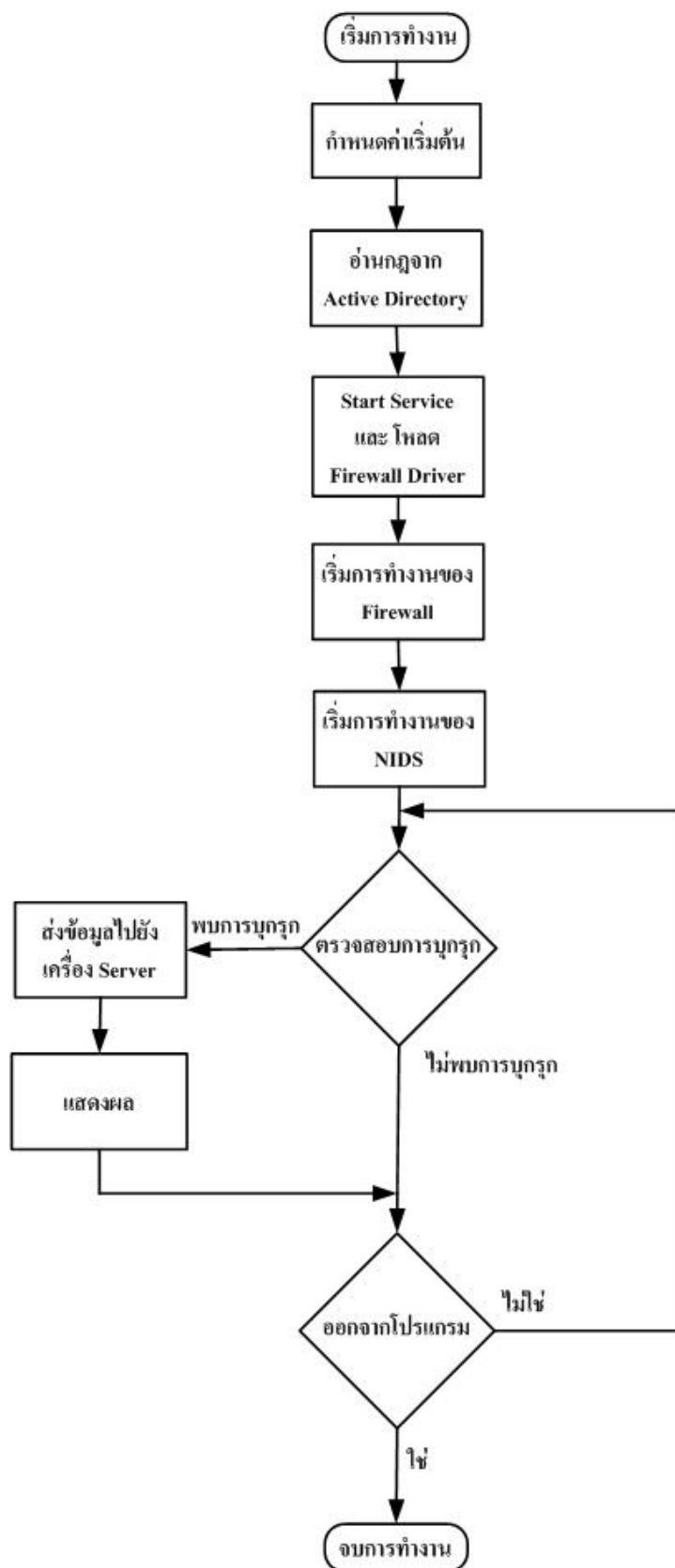
3. int CountRecord(void)

หน้าที่ : นับจำนวนข้อมูลทั้งหมดที่อยู่ในฐานข้อมูล

พารามิเตอร์ : ไม่มี

ค่ารีเทิร์น : เป็นเลขจำนวนเต็มของจำนวนข้อมูลทั้งหมดในฐานข้อมูล

คู่มือโปรแกรมเมอร์ โปรแกรม ISAG Personal Firewall



รูปที่ 3 แสดงการทำงานของโปรแกรม Personal Firewall

โปรแกรมนี้ประกอบด้วย 2 ส่วน คือส่วนของไฟร์วอลล์ กับ ส่วนของตรวจจับผู้บุกรุก ส่วนของไฟร์วอลล์ ประกอบด้วยคลาสที่สำคัญดังต่อไปนี้

1. ADConnect ทำหน้าที่ติดต่อกับ Active Directory
2. CDlgMain ทำหน้าที่จัดการเกี่ยวกับ tab หน้าหลักทั้งหมด
3. CDlgRule ทำหน้าที่จัดการเกี่ยวกับการแสดงผลกฎที่รับเข้ามาทำงาน
4. CPacketFilter ทำหน้าที่เป็นส่วนจัดการเกี่ยวกับการฟิลเตอร์แพ็คเกต
5. CRuleManage ทำหน้าที่จัดการเกี่ยวกับกฎการฟิลเตอร์ จัดเก็บกฎของโปรแกรม
6. TDriver ทำหน้าที่จัดการกับไดรเวอร์การฟิลเตอร์

คลาส ADConnect

เหมือนกับที่กล่าวมาแล้วข้างต้น

คลาส CDlgMain

ประกอบด้วยฟังก์ชันสำคัญๆ ดังต่อไปนี้

1. bool AddFilterToFw(unsigned long srcIp, unsigned long srcMask, unsigned short srcPort,

unsigned long dstIp, unsigned long dstMask, unsigned short dstPort,

unsigned int protocol, int action)

หน้าที่ : ทำการส่งกฎการฟิลเตอร์ส่งไปให้ไดรเวอร์ทำงาน

พารามิเตอร์ : พารามิเตอร์ต่างๆ จะมีความหมายเหมือนตัวแปรของไฟล์เฮดเดอร์ rules.h ที่กล่าวมาแล้วข้างต้น

ค่ารีเทิร์น : จะเป็น True เมื่อทำงานเรียบร้อย และ เป็น False เมื่อมีปัญหาในการส่งกฎให้ไดรเวอร์

2. void AddGroupRule(void)

หน้าที่ : เพิ่มกฎการฟิลเตอร์ของกลุ่มเข้าไปยังกฎของโปรแกรม

พารามิเตอร์ : ไม่มี (ใช้ค่าจากตัวแปรของคลาส)

ค่ารีเทิร์น : ไม่มี

3. CString GetUserDN(void)

หน้าที่ : เรียกเอาชื่อของโดเมนของระบบ

พารามิเตอร์ : ไม่มี

คำรีเทิร์น : เป็นสตริงของชื่อของโดเมนของระบบ

4. CString GetUser(void)

หน้าที่ : เรียกเอาชื่อของยูสเซอร์ที่ล็อกอินใช้งานอยู่ในขณะนั้น

พารามิเตอร์ : ไม่มี

คำรีเทิร์น : สตริงของชื่อยูสเซอร์ที่ล็อกอินใช้งานขณะนั้น

5. void ListMemberOf(void)

หน้าที่ : เรียกเอาชื่อของยูสเซอร์ทั้งหมดที่อยู่ในกลุ่มเดียวกัน

พารามิเตอร์ : ไม่มี

คำรีเทิร์น : ไม่มี

คลาส CDlgRule

ประกอบด้วยฟังก์ชันสำคัญๆ ดังต่อไปนี้

1. void UpdateGroupRuleList(void)

หน้าที่ : นำกฎการฟิเตอร์จากตัวแปรที่เก็บกฎนั้น ไปแสดงผลที่ลิสต์

พารามิเตอร์ : ไม่มี

คำรีเทิร์น : ไม่มี

คลาส CRuleManage

ประกอบด้วยฟังก์ชันสำคัญๆ ดังต่อไปนี้

1. void AddRule(unsigned long srcIp, unsigned long srcMask, unsigned short srcPort, unsigned long dstIp, unsigned long dstMask, unsigned short dstPort, unsigned int protocol, int action)

หน้าที่ : เพิ่มกฎเข้าไปยังตัวแปรที่เก็บกฎการฟิเตอร์

พารามิเตอร์ : พารามิเตอร์ต่างๆ จะมีความหมายเหมือนตัวแปรของไฟล์เฮดเดอร์ rules.h ที่กล่าวมาแล้วข้างต้น

คำรีเทิร์น : ไม่มี

2. void DeleteRule(unsigned int position)

หน้าที่ : ลบกฎออกจากตัวแปรที่เก็บกฎการฟิเตอร์

พารามิเตอร์ :

unsigned int position เป็นตำแหน่งลำดับที่ของกฎที่ต้องการลบออก
คำรีเทิร์น : ไม่มี

3. CString GetClientIP(void)

หน้าที่ : เรียกเอาค่าหมายเลขไอพีของเครื่องที่ลือคอินขณะนั้น

พารามิเตอร์ : ไม่มี

คำรีเทิร์น : สตริงของหมายเลขไอพีของเครื่องนั้นๆ

4. void ResetRule(void)

หน้าที่ : รีเซตกฎทั้งหมดในตัวแปรให้เป็น 0

พารามิเตอร์ : ไม่มี

คำรีเทิร์น : ไม่มี

ส่วนของการตรวจจับผู้บุกรุก

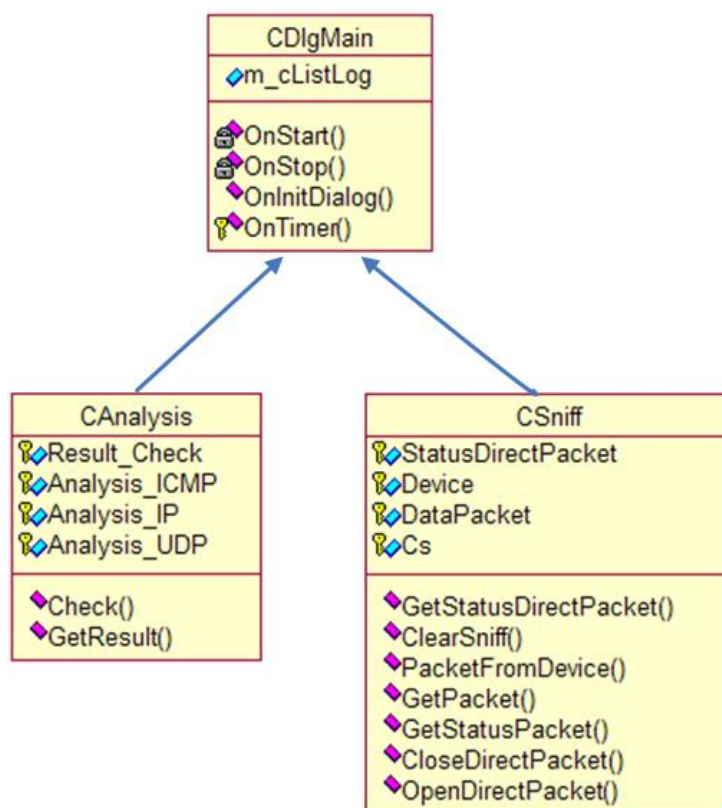
ประกอบด้วยคลาสต่างๆ ดังต่อไปนี้

1. Bomb_Packet ทำหน้าที่ตรวจสอบว่าเป็นการโจมตีลักษณะแพ็กเก็ตบอม หรือไม่
2. CAnaly_ICMP ทำหน้าที่ตรวจจับแพ็กเก็ต ICMP
3. CAnaly_IP ทำหน้าที่ตรวจจับแพ็กเก็ต IP
4. CAnalysis ทำหน้าที่ตรวจสอบว่าเป็นการโจมตีประเภทไหน
5. CAnaly_TCP ทำหน้าที่ตรวจจับแพ็กเก็ต TCP
6. CAnaly_UDP ทำหน้าที่ตรวจจับแพ็กเก็ต UDP
7. CGetFin ทำหน้าที่ตรวจจับแพ็กเก็ต Finish
8. CGetPort ทำหน้าที่ตรวจจับพอร์ตที่ใช้งาน
9. CSniff ทำหน้าที่ตรวจจับแพ็กเก็ตที่ผ่านเข้ามา
10. Fragment ทำหน้าที่ตรวจสอบการทำ Fragment
11. Header_Packet ทำหน้าที่เป็นคลาสที่ใช้เก็บข้อมูลส่วน หัว (Header) ของแต่ละแพ็กเก็ต
12. PingSweeps ทำหน้าที่ตรวจจับการ Ping Sweeps

โครงสร้าง และ การทำงานของระบบตรวจจับผู้บุกรุกทางเครือข่ายคอมพิวเตอร์

สามารถแบ่งส่วนการทำงานของโปรแกรมออกเป็น 2 ส่วนคือ

1. คลาสหลักของระบบ

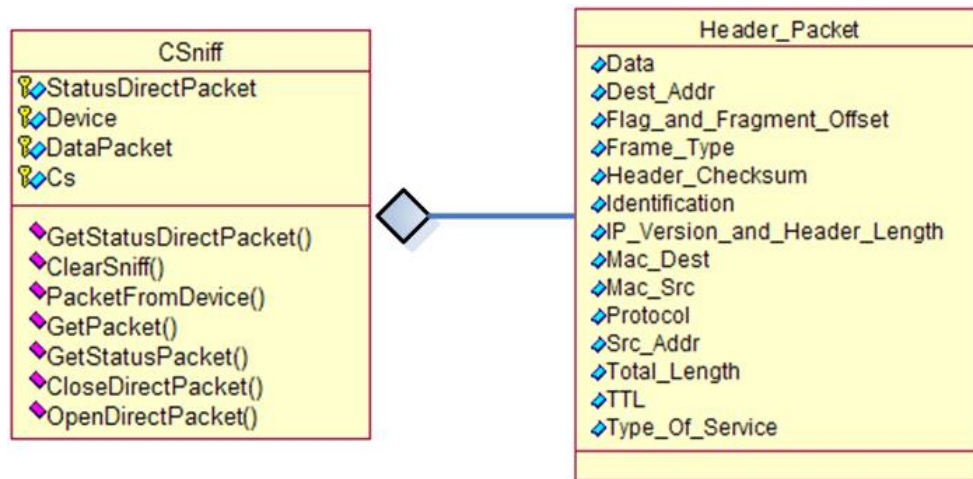


รูปที่ 4 คลาสไดอะแกรมหลักของระบบ

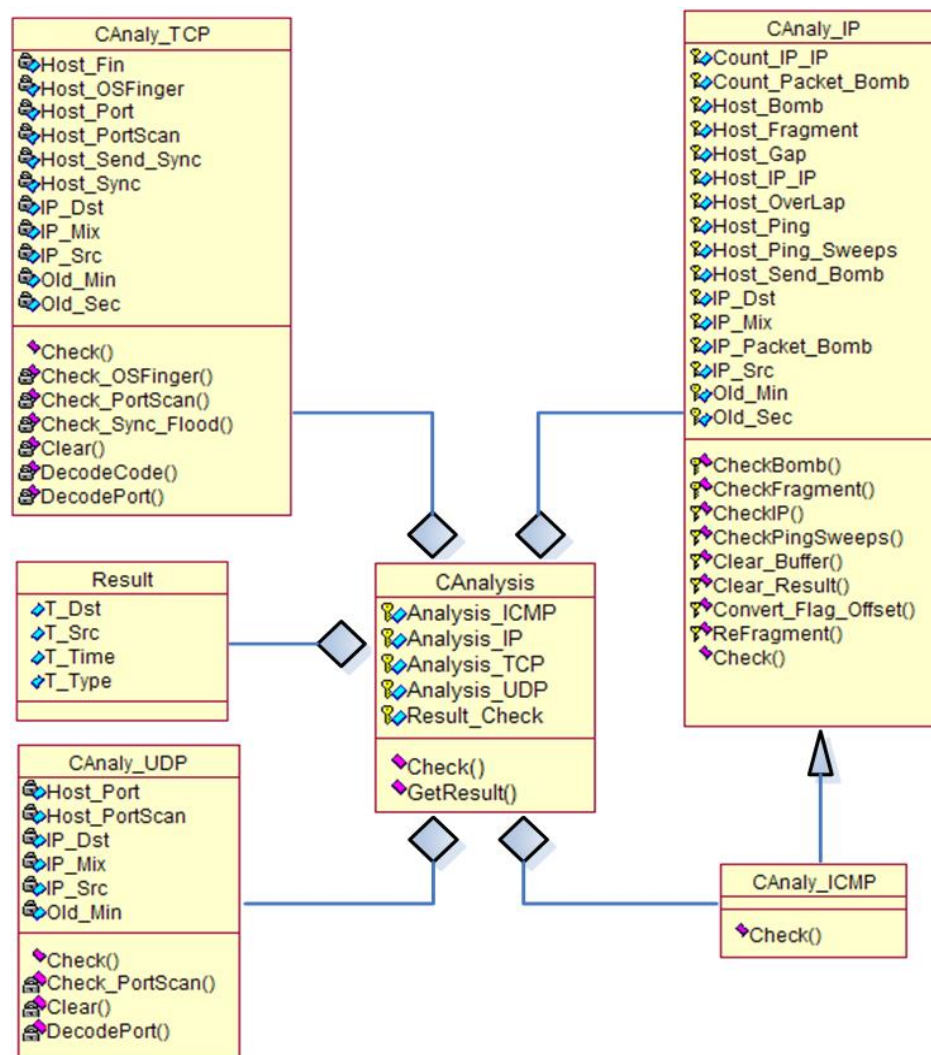
หน้าที่ของส่วนต่างๆ มีดังนี้

1. CDlgMain ทำหน้าที่ควบคุมการทำงานของระบบตรวจจับผู้บุกรุก
2. CSniff ทำหน้าที่ดักจับแพ็กเก็ตข้อมูลในเครือข่ายเก็บไว้ในบัฟเฟอร์
3. CAnalysis ทำหน้าที่วิเคราะห์ข้อมูลในบัฟเฟอร์ที่ดักจับมาได้

2. คลาสไดอะแกรมส่วนตรวจจับแพ็กเก็ต และ วิเคราะห์การบุกรุก



รูปที่ 5 คลาสไดอะแกรมส่วนที่ทำการดักจับข้อมูล



รูปที่ 6 คลาสไดอะแกรมส่วนวิเคราะห์การบุกรุก

4.4.1 รายละเอียดการทำงานของโปรแกรมแต่ละส่วน

ฟังก์ชันการทำงานแต่ละส่วนสามารถแบ่งตามคลาสได้ดังนี้

1. คลาส CDlgMain ควบคุมการทำงานทั้งหมดของระบบตรวจจับผู้บุกรุก

ทำหน้าที่กำหนด ค่าต่างๆ และควบคุมส่วนต่างๆ ให้ทำงานร่วมกัน ในส่วนของการควบคุม จะใช้คลาส CDlgMain เพื่อเริ่มการทำงานดังต่อไปนี้

ขั้นตอนการทำงาน

1. ตรวจสอบว่ามีการสั่งหยุดการทำงานหรือยัง
2. นำข้อมูลออกมาจากบัฟเฟอร์ของคลาส CSniff
3. เคลียร์ลิสต์ของแพ็กเก็ตในกรณีที่มีการโจมตี โดยวิธี รีเฟรชเมนต์แบบ Gap

โดยฟังก์ชัน ClearSniff() ของคลาส CSniff

- void CDlgMain::OnStart()

จุดมุ่งหมาย

เป็นส่วนที่ใช้ในการเริ่มการทำงานของเทรดต่างๆ และสั่งให้เทรดเริ่มการทำงาน

ขั้นตอนการทำงาน

1. ตรวจสอบว่ามีการเปิดโหมดการทำงานเป็นไคเร็กซ์ของการ์ดแลนหรือยัง
2. ทำการเปิดโหมดการทำงานเป็นไคเร็กซ์ของการ์ดแลน
3. เริ่มการทำงานของ Thread_Sniff()
4. เริ่มการทำงานของ Thread_Analysis()

- void CDlgMain::OnStop()

จุดมุ่งหมาย

เป็นส่วนที่ใช้ในการหยุดการทำงานของเทรด และ ลบข้อมูลออกจากบัฟเฟอร์

ขั้นตอนการทำงาน

1. ตรวจสอบว่ามีการเปิดโหมดไคเร็กซ์ของการ์ดแลน หรือ ไม่ถ้ามีการเปิดจะทำตามขั้นตอนดังนี้คือ
2. สั่งให้ Thread_Sniff() หยุดการทำงาน
3. สั่งให้ Thread_Analysis() หยุดการทำงาน
4. ทำการเคลียร์ข้อมูลในบัฟเฟอร์ของการ์ดแลน
5. ทำการปิดโหมดการทำงานของการ์ดแลน

- `UINT Thread_Sniff()`

จุดมุ่งหมาย

เป็นส่วนที่ใช้เก็บข้อมูลที่เป็นของเครื่องตนเก็บไว้ในบัฟเฟอร์โดยการทำงานในส่วนนี้จะทำงานเป็นเทรดแยกออกมาจากฟังก์ชันหลัก

ขั้นตอนการทำงาน

1. เช็คว่ามีการสั่งหยุดการทำงานหรือยัง
2. สั่งให้ทำการเก็บข้อมูลจากการ์ดแลน ด้วยฟังก์ชัน `PacketFromDevice()` ของคลาส `CSniff`

- `UINT Thread_Analysis()`

จุดมุ่งหมาย

เป็นส่วนที่ใช้ในการวิเคราะห์แพ็กเก็ตข้อมูลที่อยู่ในบัฟเฟอร์ที่รับเข้ามาจากการ์ดเน็ตเวิร์ค โดยการทำงานในส่วนนี้จะทำงานเป็นเทรดแยกออกมาจากฟังก์ชันหลัก

4.4.2 การเก็บข้อมูล

ในการเก็บข้อมูลจากเน็ตเวิร์คมาทำการวิเคราะห์ โปรแกรมจะเก็บข้อมูลส่วนหัว (Header) ของแต่ละแพ็กเก็ต ในชั้นต่างๆ เช่นเก็บเฉพาะข้อมูลส่วนหัวของแพ็กเก็ตในชั้นไอพี และทีซีพี ในการเก็บข้อมูลจะใช้คลาส `Csniff` เป็นตัวเก็บข้อมูลโดยมีฟังก์ชันการทำงานดังต่อไปนี้

- `BOOL OpenDirectPacket(int NumberDevice)`

จุดมุ่งหมาย

เพื่อทำการเปิดไดเร็กต์แพ็กเก็ตโหมดของการ์ดแลน

ขั้นตอนการทำงาน

1. เช็คว่ามีการเปิดไดเร็กต์แพ็กเก็ตโหมดของการ์ดแลนอยู่หรือไม่
2. ทำการเปิดไดเร็กต์แพ็กเก็ตโหมดของการ์ดแลนตามค่า `NumberDevice` ที่รับเข้ามา

- `BOOL CloseDirectPacket()`

จุดมุ่งหมาย

เพื่อทำการปิดไดเร็กต์แพ็กเก็ตโหมดของการ์ดแลน

ขั้นตอนการทำงาน

1. เช็คว่ามีการเปิดไดเร็กต์แพ็กเก็ตโหมดของการ์ดแลนอยู่หรือไม่
2. ทำการปิดไดเร็กต์แพ็กเก็ตโหมดของการ์ดแลนที่เปิดอยู่

- **BOOL GetStatusPacket()**

จุดมุ่งหมาย

เพื่อตรวจสอบว่ามีการรับแพ็กเก็ตเข้ามาเท่าไร แล้วมีการสูญหายไปเท่าไร

ขั้นตอนการทำงาน

1. เช็คว่ามีการเปิดไดเร็กแพ็กเก็ตโหมดของการ์ดแลนอยู่หรือไม่
2. ทำการดึงค่าขึ้นมาจาก Winpcap

- **BOOL GetPacket()**

จุดมุ่งหมาย

เพื่อนำค่าแฮดเดอร์ของแพ็กเก็ตที่เก็บไว้ส่งออกไปให้ฟังก์ชันอื่นทีละ 1 แพ็กเก็ต

ขั้นตอนการทำงาน

1. เช็คว่ามีแพ็กเก็ตอยู่ในลิงค์ลิสต์หรือไม่
2. อ่านค่าใน head ของลิงค์ลิสต์ออกมา
3. ลบส่วนหัวของลิงค์ลิสต์ทิ้ง

- **BOOL PacketFromDevice()**

จุดมุ่งหมาย

เพื่อนำข้อมูลแฮดเดอร์แพ็กเก็ตในการ์ดแลนออกมา แล้วนำมาเข้าลิงค์ลิสต์

ขั้นตอนการทำงาน

1. เช็คว่ามีการเปิดไดเร็กแพ็กเก็ตโหมดของการ์ดแลนอยู่หรือไม่
2. จัดการนำข้อมูลในบัพเฟอร์ของการ์ดแลนออกมา
3. ทำการแยกออกเป็นแพ็กเก็ต โดยจะแยกเอาเฉพาะส่วนหัวของแพ็กเก็ต
4. นำเข้าไปเก็บในลิงค์ลิสต์โดยที่ 1 โหนด จะเก็บ 1 แพ็กเก็ต

- **BOOL ClearSniff()**

จุดมุ่งหมาย

เพื่อลบข้อมูล ในลิงค์ลิสต์ทั้งหมด

ขั้นตอนการทำงาน

1. จะทำการลบข้อมูลในลิงค์ลิสต์

- **BOOL GetStatusPromiscuous()**

จุดมุ่งหมาย

เพื่อเช็คว่ามีการเปิดไดเร็กแพ็กเก็ตโหมดของการ์ดแลนหรือไม่

ขั้นตอนการทำงาน

1. เช็คว่ามีการเปิดไคเร็กแพ็กเก็ตโจมตีของการ์ดแลนอยู่หรือไม่
2. ส่งผลลัพธ์กลับสู่ฟังก์ชันหลักว่าเปิดหรือไม่

4.4.3 การวิเคราะห์ข้อมูล

ในส่วนของการวิเคราะห์ข้อมูล จะใช้คลาสที่ทำหน้าที่วิเคราะห์ข้อมูลคือคลาส CAnalysis, คลาส CAnaly_IP, คลาส CAnaly_ICMP, คลาส CAnaly_TCP, และคลาส CAnaly_UDP โดยมีฟังก์ชันการทำงานดังต่อไปนี้

- **BOOL CAnaly_IP::Check()**

จุดมุ่งหมาย

เพื่อเช็คแพ็กเก็ตที่ได้รับมา แล้วรายงาน

ขั้นตอนการทำงาน

1. ส่งแพ็กเก็ตที่ได้รับมาไปวิเคราะห์ เพื่อแจ้งผล
2. ตรวจสอบว่าแพ็กเก็ตไหนที่ยังไม่สามารถรีเฟรชเมนต์ได้ในระยะเวลาที่กำหนด เพื่อแจ้งผล

- **BOOL CAnaly_ICMP::Check()**

จุดมุ่งหมาย

เพื่อเช็คแพ็กเก็ตที่ได้รับมา แล้วรายงาน

ขั้นตอนการทำงาน

1. ส่งแพ็กเก็ตที่ได้รับมาไปวิเคราะห์ เพื่อแจ้งผล

- **BOOL CAnalysis::Check()**

จุดมุ่งหมาย

เพื่อแยกชนิดของแพ็กเก็ตว่าเป็นชนิดไหนเพื่อจะได้นำไปวิเคราะห์ ต่อไป

ขั้นตอนการทำงาน

1. นำแพ็กเก็ตที่ได้มาตรวจสอบว่าเป็นประเภทใด
2. นำไปทำการวิเคราะห์ ตามประเภทของแพ็กเก็ต
3. ทุกประเภทจะต้องผ่านการวิเคราะห์ จากหมายเลขไอพีก่อน ยกเว้นไอซีเอ็มพี ซึ่งจะ มีการวิเคราะห์ ที่แยกออกไป แต่เป็นการสืบทอดมาจากวิเคราะห์แบบหมายเลขไอพี

- **BOOL CAnaly_IP::CheckBomb() และ BOOL CAnaly_ICMP::CheckBomb()**

จุดมุ่งหมาย

เพื่อตรวจสอบว่ามีแพ็กเก็ตเกิดจากหมายเลขไอพีหนึ่ง ไปยังอีกหมายเลขไอพีหนึ่งมากเกินไปหรือไม่ ในเวลาที่เหมาะสม

ขั้นตอนการทำงาน

1. นำแพ็กเก็ตที่ได้มาดูว่ามาจากหมายเลขไอพีไหน แล้วส่งไปยังหมายเลขไอพีไหน
2. แล้วทำการนับว่าเกินกว่าที่กำหนดหรือไม่ (2000 แพ็กเก็ต / วินาที)
3. ถ้าครบจะมีการเปรียบเทียบกับข้อมูลของการโจมตีแบบนี้สำหรับหมายเลขไอพีนี้ ว่ามีการโจมตีแบบนี้ไปแล้วหรือยัง เพื่อจะได้ไม่มีการแจ้งเตือนเกินไป

- BOOL CAnaly_IP::CheckIP() และ BOOL CAnaly_ICMP::CheckIP()

จุดมุ่งหมาย

เพื่อตรวจสอบว่ามีแพ็กเก็ตเกิดจากหมายเลขไอพีต้นทางไปยังหมายเลขไอพีปลายทางเป็นหมายเลขไอพีเดียวกันหรือไม่

ขั้นตอนการทำงาน

1. นำแพ็กเก็ตที่ได้มาดูว่ามาจากหมายเลขไอพีไหน แล้วส่งไปยังหมายเลขไอพีไหน
2. ดูว่าหมายเลขไอพีตรงกันหรือไม่
3. ถ้าตรงกันจะมีการเปรียบเทียบกับข้อมูลของการโจมตีแบบนี้สำหรับหมายเลขไอพีนี้ ว่ามีการโจมตีแบบนี้ไปแล้วหรือยัง เพื่อจะได้ไม่มีการแจ้งเตือนเกินไป

- BOOL CAnaly_IP::CheckFragment() และ BOOL CAnaly_ICMP::CheckFragment()

จุดมุ่งหมาย

เพื่อแยกแพ็กเก็ตตามหมายเลขไอพีต้นทาง, หมายเลขไอพีปลายทาง, โพรโทคอลและฟิลด์ Identifier ตรวจสอบว่ามีแพ็กเก็ตเกิดจากหมายเลขไอพีต้นทางไปยังหมายเลขไอพีปลายทางต้องการทำแฟร็กเมนต์หรือไม่

ขั้นตอนการทำงาน

1. เชื่อว่าแพ็กเก็ตที่ได้รับมาเป็นแพ็กเก็ตที่จะต้องทำแฟร็กเมนต์หรือไม่
2. จะดูว่ามีข้อมูลเดิมอยู่หรือไม่โดยจะดูที่หมายเลขไอพีต้นทาง ,หมายเลขไอพีปลายทาง ,โพรโทคอลและ Identification
3. นำเข้าสู่ลิสต์

- BOOL CAnaly_IP::ReFragment() และ CAnaly_ICMP::ReFragment()

จุดมุ่งหมาย

เพื่อตรวจสอบว่าแพ็กเก็ตมีปัญหาเรื่องการแฟร็กเมนต์หรือไม่

ขั้นตอนการทำงาน

1. เชื่อกันว่าในแพ็กเก็ตที่ได้รับมาแล้วนั้นมีส่วนหัวของแพ็กเก็ตและส่วนท้ายของแพ็กเก็ตหรือไม่
2. จะลองประกอบดูว่าสามารถประกอบได้หรือไม่
3. ถ้าประกอบได้จะลบออกจากลิสต์

- void CAnaly_IP::Clear() and void CAnaly_ICMP::Clear()

จุดมุ่งหมาย

เพื่อเคลียร์ข้อมูลการโจมตี

ขั้นตอนการทำงาน

1. ทำการเคลียร์ข้อมูลการโจมตี